

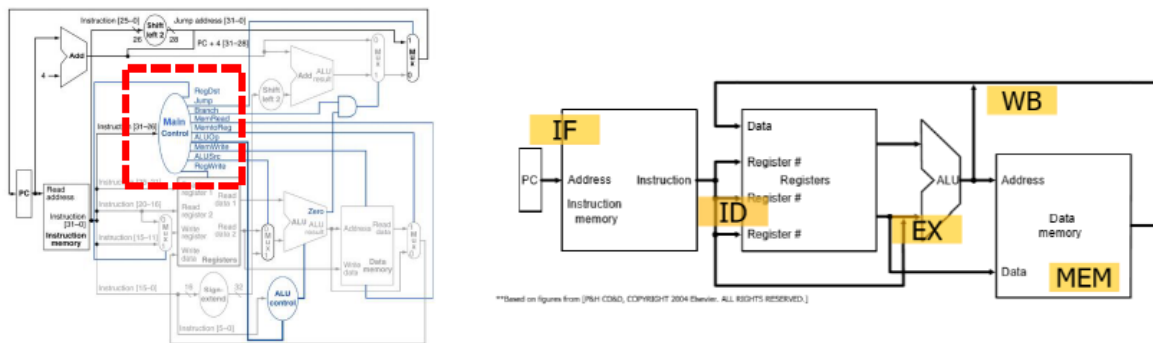
Introduction

이번 랩에서는 single cycle CPU를 Verilog를 활용하여 구현하는 것이 목표이다. single cycle CPU 의 구성요소들과 그 작동방식을 이해하는 것이 이번 랩의 주 목표이다. CPU는 크게 두 구성요소로 이루어지는데, 바로 data path와 control이다. Data path는 system bus에서 ALU, register file 그리고 다시 system bus로 돌아오는 path를 의미하며, control unit은 instruction별로 어떠한 data path를 따라가야 하는지 조정하는 역할을 한다. 우리가 구현하는 CPU는 하나의 instruction을 각 clock cycle마다 process하며, RISC-V CPU보다 더 짧고 적은 instruction과 작은 register를 가진 TSC CPU이다. 개념들을 잘 이해하고 CPU의 구성요소를 module화 하여 코드를 작성해야 한다.

Design

Combinational logic과 Sequential logic을 이용해 작동하는 CPU를 구현했다. 대부분의 계산은 register와 ALU를 연결하여 이루어지도록 하였으며, module은 CPU와 ALU 두 개로 나누었다.

Control Unit



cpu에서 instruction processing은 IF, ID, EX, MEM, WB의 다섯 가지 단계로 이루어지므로,

clk과 Inputready, Ackoutput 변수를 이용해 always 문을 구성했다.

Instruction format



Data를 통해 읽을 16-bit instruction format은 다음과 같았다.

Opcode, rs, rt, rd, function, offset, target address를 일단 모두 읽은 뒤, opcode와 function code에 따라 맞는 format을 결정하고, 필요한 동작을 하도록 구현했다.

Implementation

```
module cpu (readM, writeM, address, data, ackOutput, inputReady, reset_n, clk);
    output reg readM; // "read" signal to memory
    output reg writeM; // "write" signal to memory
    output reg [`WORD_SIZE-1:0] address; // target memory address
    inout [`WORD_SIZE-1:0] data; // data for reading or writing (can be used as both input and output)
    input ackOutput; // signal from memory ("data is written")
    input inputReady; // signal from memory ("data is ready for reading")
    input reset_n; // reset your CPU (registers, PC, etc)
    input clk; // clock signal

    reg [`WORD_SIZE-1:0] pc; // program counter

    reg [3:0] Op_Code; // operation code
    reg [1:0] rs, rt, rd, wr; // register index
    reg [`WORD_SIZE-1:0] regs[3:0]; // register file
    reg [5:0] Func_Code; // function code
    reg [7:0] Immediate; // immediate value
    reg [`WORD_SIZE-1:0] Sign_Extend_Immediate; // sign extension
    reg [11:0] Target_Address; // jump address

    reg [`WORD_SIZE-1:0] A, B; // ALU operand
    wire [`WORD_SIZE-1:0] C; // ALU result
    wire bcond; // ALU branch condition

    reg load_check; // check if instruction was 'LWD_OP' or not

    reg Register_Write_Condition; // write condition boolean true or false
    reg [`WORD_SIZE-1:0] Write_Back_Register; // write register value

    // If store, put register value of reg[rt] in memory, otherwise read data
    assign data = (writeM == 1) ? regs[rt] : `WORD_SIZE'bz;

    // initialize
```

초기 변수 설정은 다음과 같이 해주었다.

Instruction을 읽어야 할 변수들과, ALU를 통해 Branch condition을 전달받을 bcond 변수, 그리고 load mode일 경우를 체크하기 위한 load_check 등이 선언되었다.

이 때, data는 writeM == 1 일 경우 쓰기 모드로 동작하고, 나머지 경우에 데이터를 읽어올 수 있도록 assign 문을 선언해주었다.

전체적인 clock의 흐름이

Clock+, readM+ => readM-, inputReady+ => inputReady- => clock- => Clock+, readM+ ...

를 따르도록 하고, always문을 이용해 이 사이에 5단계의 cpu 동작이 연결되도록 구현해주었다.

(여기서 +, -는 각각 posedge, negedge를 의미한다.)

다음은 readM- 일 경우, 읽어온 data 값을 가지고 register 연산을 하는 내용의 코드이다.

Op_Code와 Func_Code에 따라 케이스를 나누어 주었다.

이 때, Sign_Extension을 할 때, 8bit offset을 16bit로 extension 해주어야 하므로, shift 연산을 사용하지 않고 케이스를 나누어서 Sign_Extend_Immediate에 8bit를 덧붙여주었다.

```

//Data read from memory -> Carry out instructions according to types.
always @(negedge readM) begin

    if(load_check == 0) begin
        if( Op_Code == `ALU_OP) begin //R-type (RWD WWD none)
            A = regs[rs];
            B = regs[rt];
            wr = rd;
            pc = pc + 1;
            end

        else if ( Op_Code == `JMP_OP) begin //J-type (JAL JPR JRL none)
            pc = Target_Address;
            end

        else begin //I-type
            pc = pc + 1;
            if(Immediate[7] == 1) begin//If MSB is 1 (negative value)
                Sign_Extend_Immediate = {8'hff, Immediate}; //concatenate 1111 1111
            end
            else Sign_Extend_Immediate = {8'h00, Immediate}; //MSB is 0, concatenate 0000 0000

            if(Op_Code >= `BNE_OP && Op_Code <= `BLZ_OP) begin //Branch operation, check bcond
                A = regs[rs];
                B = regs[rt];
                end

            else if (Op_Code >= `ADI_OP && Op_Code <= `LHI_OP) begin // Immediate type
                A = regs[rs];
                B = Sign_Extend_Immediate;
                Write_Back_Register = C;
                wr = rt;
                end

            else begin//Store or Load type
                address = regs[rs] + Sign_Extend_Immediate;//store
                if(Op_Code == `LWD_OP) begin //load
                    readM = 1;
                    load_check = 1;
                end
            end
        end
    end
end

```

대부분의 연산은 ALU에서 이루어지고, 결과값을 'C'라는 wire Result에 output 하도록 설정해두었다.

Discussion

저번 랩 과제보다 난이도가 갑자기 높아진 것 같아서 구현하는데 힘겨웠다, 특히 정해진 매뉴얼이나 단계가 없는 상태로 구현을 시작해서 처음에 많이 헤맸던 것 같다. 앞으로는 구현에 앞서 보다 더 정교하게 설계를 한 후 구현을 시작해야 할 것 같다.

지금은 간단한 CPU를 구현하는 것이라서 module이 CPU와 ALU로만 구성되어도 큰 문제는 없었지만, 만약 앞으로 더 복잡한 CPU를 구현하게 된다면, 효율적인 설계를 위해 module화를 좀 더 해야겠다고 느꼈다.

Conclusion

결론적으로, 우리는 single cycle TSC CPU를 구현할 수 있었으며, test bench가 요구하는 사항을 모두 충족시킬 수 있었다. 이번 랩을 통해서 single cycle CPU의 작동원리에 대해서 배울 수 있었다.